**PROJETO MAYA APP Grupo**

**DevMasters**

**Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas**

**INTEGRANTES DO PROJETO e RA'S**

**Pedro Henrique Araujo Del Picolo - 25027316**
**Matheus Fernandes Moraes. - 25026099**
**Ana luiza ribeiro. - 24026973**
**Henri Seixas Souza - 25027353**

São Paulo

2025

## Sumário

1. Qualidade de Software ..... 6

1.1. Modelo que qualidade de software (Diagrama/Design) ..... 6

1.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo) ..... 6

1.3. Identificação de atributos de qualidade da norma 25010. ..... 6

1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento. ..... 6

2. Teste de Software ..... 6

2.1. Plano de Teste ..... 6

2.2. Apresentar 2 testes unitários. ..... 6

2.3. Apresentar 2 testes de integração ..... 6

2.4. Apresentar um teste de usuário (SISTEMA) ..... 7

4. REFERÊNCIAS BIBLIOGRÁFICAS ..... 7

| Data das entregas | 30/mar. | 11/mai. |
|---|---|---|
| Disciplinas | Entrega 1 (3,0 Pontos) | Entrega 2 (4,0 Pontos) |
| Qualidade de Software ( | **Instruções de Entrega: Qualidade de Software**<br>Objetivo: O aluno deverá demonstrar o entendimento dos conceitos de qualidade de software, aplicando-os ao projeto. Isso inclui a criação de um diagrama do processo de qualidade de software, a identificação de atributos de qualidade. Apresentar um relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento. | **Instruções de Entrega:**<br>**Testes Unitários e de Integração**<br>Objetivo: Os alunos deverão desenvolver 4 testes unitários e 2 testes de integração para o sistema desenvolvido. Esses testes devem garantir que o sistema esteja funcionando corretamente e validando os principais componentes e integrações e a geração de um relatório de teste de carga do servidor.<br>**Teste de Sistema / Aceitação**<br>Objetivo: O aluno deverá criar um teste de sistema ou aceitação para verificar se o sistema como um todo está funcionando conforme o esperado. O teste de aceitação verifica se as funcionalidades implementadas atendem aos requisitos do cliente ou usuário final, garantindo que o sistema está pronto para ser utilizado em produção. |

# ORIENTAÇÃO PARA OS TÓPICOS

## 1. INTRODUÇÃO

Na introdução, o grupo deve apresentar o contexto do projeto, explicando qual problema será resolvido, qual é a proposta do sistema e qual sua relevância acadêmica e prática. É importante descrever brevemente o público-alvo, os objetivos do sistema e as tecnologias que serão utilizadas.

Além disso, a introdução deve delimitar o escopo do projeto, deixando claro o que será desenvolvido e o que não faz parte da proposta. O texto deve preparar o leitor para entender como qualidade e testes serão aplicados ao longo do desenvolvimento.

---

## 2. Qualidade de Software

Neste tópico, o grupo deve explicar o que significa qualidade para o sistema desenvolvido e por que ela é essencial no projeto. A qualidade pode envolver aspectos como desempenho, segurança, usabilidade, confiabilidade e manutenibilidade.

Recomenda-se fundamentar o conceito com base na norma ISO/IEC 25010, explicando que a qualidade será avaliada tanto nos requisitos funcionais quanto não funcionais. Também é interessante mencionar como a equipe pretende medir e garantir essa qualidade.

---

### 2.1. Modelo de Qualidade de Software (Diagrama/Design)

Neste item, deve ser apresentado um diagrama que represente visualmente o modelo de qualidade adotado no projeto. O grupo pode criar um diagrama relacionando atributos de qualidade (como segurança, desempenho e usabilidade) aos requisitos do sistema.

O objetivo é demonstrar, de forma gráfica, como cada atributo será tratado no design da solução. Pode ser feito em ferramentas como draw.io e deve mostrar claramente a ligação entre requisitos, métricas e validações.

---

### 2.2. Processo (Plano) de Gerenciamento de Qualidade de Software

Aqui deve ser descrito, em texto explicativo, como a qualidade será planejada, controlada e monitorada ao longo do projeto. O grupo deve explicar quais práticas serão adotadas, como

versionamento (Git), revisão de código, testes automatizados, validação com usuário e controle de erros.

Também é importante definir responsabilidades da equipe, critérios de aceite e como serão tratadas não conformidades. O plano deve mostrar organização e preocupação com melhoria contínua.

---

**2.3. Identificação de Atributos de Qualidade da Norma 25010**

Neste tópico, o grupo deve identificar quais atributos da norma ISO/IEC 25010 são mais relevantes para o sistema proposto. Entre eles estão: funcionalidade, confiabilidade, usabilidade, eficiência de desempenho, compatibilidade, segurança, manutenibilidade e portabilidade.

Cada atributo selecionado deve ser brevemente explicado e contextualizado dentro do projeto, mostrando por que ele é importante para o sistema desenvolvido.

---

**2.4. Relatório explicando como a norma 25010 é utilizada no processo de desenvolvimento**

Aqui o grupo deve elaborar um pequeno relatório explicando como os atributos da norma foram incorporados ao ciclo de desenvolvimento. Deve-se mostrar como os requisitos não funcionais foram definidos com base na norma e como eles são verificados por meio de testes e validações.

O texto deve demonstrar que a norma não foi apenas citada, mas realmente aplicada como referência para definição de métricas, critérios de qualidade e melhoria do sistema.

---

**3. Teste de Software**

Neste tópico, o grupo deve explicar a importância dos testes para garantir a qualidade do sistema. É necessário apresentar a estratégia geral de testes adotada no projeto, destacando que eles serão realizados em diferentes níveis: unitário, integração e usuário.

O objetivo é demonstrar que o sistema foi validado de forma estruturada antes da entrega final.

---

**3.1. Plano de Teste**

O plano de teste deve descrever quais tipos de testes serão realizados, quais ferramentas serão utilizadas (ex.: JUnit, pytest, etc.), quem será responsável pelos testes e em que momento do cronograma eles ocorrerão.

Também devem ser definidos critérios de sucesso, critérios de falha e procedimentos para correção de erros encontrados.

---

### 3.2. Apresentar 2 Testes Unitários

Neste item, o grupo deve apresentar dois exemplos de testes unitários aplicados a funções ou métodos específicos do sistema. Cada teste deve conter: objetivo, entrada utilizada, resultado esperado e resultado obtido.

O foco é demonstrar que partes isoladas do código foram verificadas individualmente para garantir seu funcionamento correto.

---

### 3.3. Apresentar 2 Testes de Integração

Aqui devem ser apresentados dois testes que validem a interação entre módulos do sistema (por exemplo, cadastro + banco de dados, ou login + autenticação).

Cada teste deve explicar o cenário, os componentes envolvidos, o fluxo executado e o resultado esperado. O objetivo é demonstrar que os módulos funcionam corretamente quando integrados.

---

### 3.4. Apresentar um Teste de Usuário (Sistema)

Neste tópico, deve ser apresentado um teste realizado com usuário real ou simulado. É necessário descrever o perfil do usuário, o cenário de uso e as tarefas solicitadas (ex.: realizar cadastro, efetuar login, gerar relatório).

O grupo deve relatar os resultados obtidos, dificuldades encontradas e possíveis melhorias identificadas. Esse teste demonstra validação prática da usabilidade do sistema.

---

## 4. REFERÊNCIAS BIBLIOGRÁFICAS

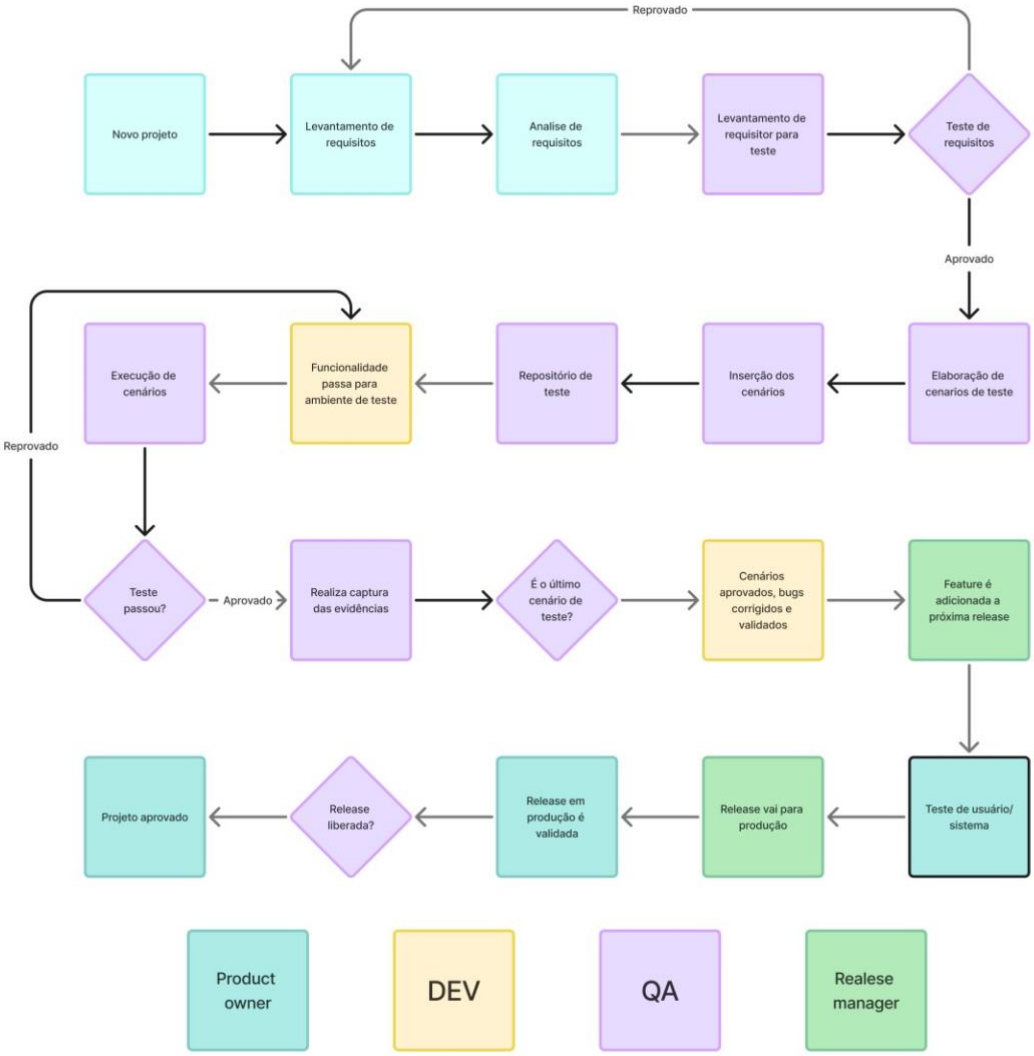
Neste item, devem ser listadas todas as fontes utilizadas na fundamentação teórica do trabalho. As referências devem seguir o padrão exigido pela instituição (normalmente ABNT).

Devem ser incluídas normas como a ISO/IEC 25010, livros de Engenharia de Software.

# Desenvolva abaixo o seu PI

# 1. Qualidade de Software

## 1.1. Modelo que qualidade de software (Diagrama/Design)




## 1.2. Processo (plano) de gerenciamento de qualidade de software

O gerenciamento da qualidade é aplicado em todas as etapas do ciclo de vida do desenvolvimento para garantir que o sistema seja confiável e atenda aos usuários. O plano consiste em:

- **Definição de Requisitos:** Utilização da norma para identificar requisitos não funcionais, como segurança e desempenho.
- **Projeto e Arquitetura:** As características de qualidade orientam decisões técnicas que impactam o produto final.

- **Desenvolvimento e Testes:** A norma serve como referência para validar o sistema através de testes funcionais, de segurança, usabilidade e desempenho.
- **Pós-entrega:** Auxílio na manutenção e evolução contínua do software.

### 1.3. Identificação de atributos de qualidade da norma 25010.

Foram identificados as oito características principais da norma ISO/IEC 25010 para o projeto:

- **Adequação funcional:** Garante que as funções atendam às necessidades dos usuários.
- **Eficiência de desempenho:** Avalia o tempo de resposta e o uso de recursos.
- **Compatibilidade:** Capacidade de operar com outros sistemas.
- **Usabilidade:** Foco na facilidade de aprendizado e utilização.
- **Confiabilidade:** Capacidade de operar corretamente por períodos determinados.
- **Segurança:** Proteção de dados e prevenção de acessos não autorizados.
- **Manutenibilidade:** Facilidade de modificar e evoluir o sistema.
- **Portabilidade:** Capacidade de funcionar em diferentes ambientes.

### 1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.

A norma ISO/IEC 25010 é utilizada como o padrão internacional para orientar o desenvolvimento e a avaliação da qualidade. Ela substitui padrões anteriores, introduzindo melhorias em segurança e compatibilidade. No processo, a norma permite que a equipe identifique e melhore atributos técnicos enquanto avalia a experiência do usuário através dos modelos de **qualidade em uso** e **qualidade do produto**. Sua aplicação sistemática assegura que o software final esteja alinhado a critérios reconhecidos mundialmente, resultando em um produto mais seguro, eficiente e com maior satisfação do usuário.

## 2. Teste de Software

### 2.1. Plano de Teste

### 2.2. Apresentar 2 testes unitários.

### 2.3. Apresentar 2 testes de integração

### 2.4. Apresentar um teste de usuário (SISTEMA)

## 4. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.